

实验三 外部中断按键控制蜂鸣器

3.1 实验目的

外部中断是单片机常用的输入检测方式之一。当外部引脚电平发生变化时，CPU 可以暂时停止当前正在执行的程序，转而进入中断服务函数处理外部事件。本实验通过两个按键触发外部中断，并控制无源蜂鸣器发出不同频率的声音，使学生理解 STM32F407 外部中断的基本配置方法和应用场景。

3.2 实验简介

3.2.1 STM32F407 外部中断简介

在普通 GPIO 输入实验中，CPU 需要不断读取按键引脚的电平状态，这种方式称为轮询。轮询方式程序简单，但 CPU 需要不断检查按键是否被按下，效率较低。

外部中断方式不同。当按键引脚电平发生变化时，STM32 的 EXTI 模块会自动检测该变化，并向 CPU 提出中断请求。CPU 响应中断后，进入对应的中断服务函数，执行按键事件处理程序。这样即使主程序正在执行其他任务，也可以及时响应按键输入。

STM32F407 的外部中断主要涉及以下几个部分：

- （1）GPIO：用于连接按键输入引脚。
- （2）SYSCFG：用于将具体 GPIO 端口映射到对应 EXTI 中断线。
- （3）EXTI：用于配置外部中断线、触发方式和中断模式。
- （4）NVIC：用于管理中断优先级和中断使能。
- （5）中断服务函数：当中断发生后，CPU 自动跳转执行的函数。

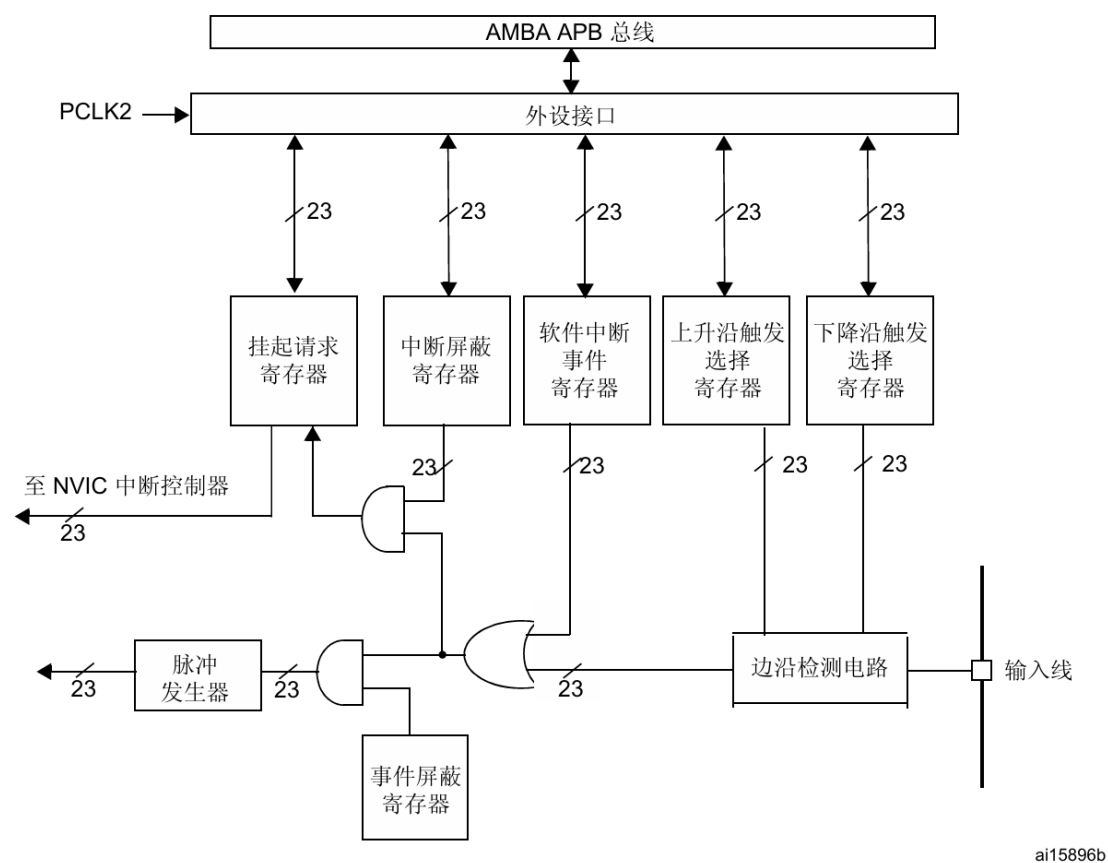


图 3.1 外部中断/事件控制器框图

3.2.2 按键输入原理

本实验中，按键采用上拉输入方式。上拉输入的特点是：按键未按下时，GPIO 引脚被上拉到高电平；按键按下时，引脚与 GND 连接，电平变为低电平。

如果只想在按键按下瞬间触发中断，可以配置为下降沿触发。如果既要检测按下，又要检测松开，可以配置为上升沿和下降沿均触发。本实验为了实现“按下蜂鸣、松开停止”，建议将按键外部中断配置为上升沿和下降沿均触发。

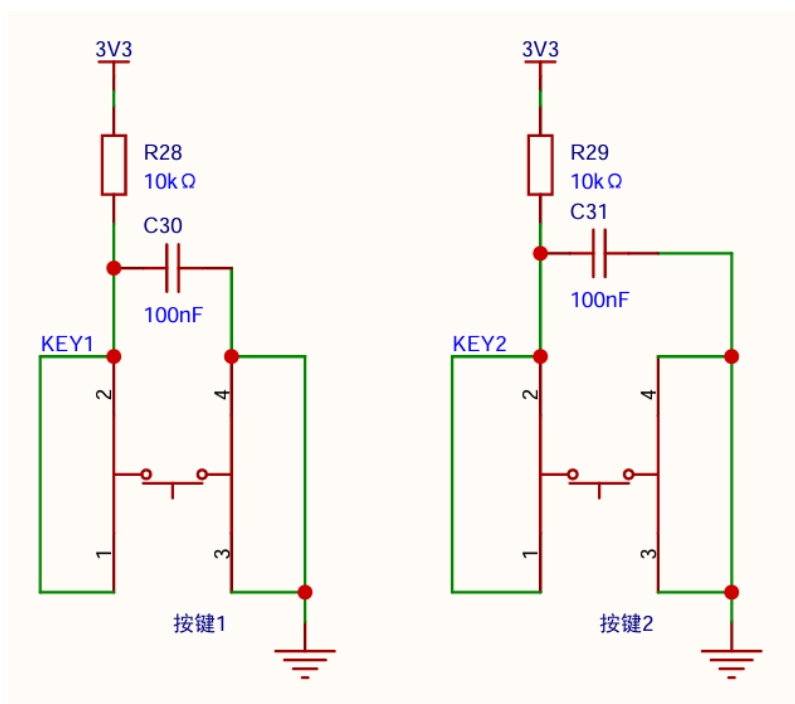


图 3.2 外部中断/事件控制器框图

3.2.3 按键消抖原理

机械按键在按下或松开的一瞬间，触点不会立即稳定，而是会产生短时间的抖动。单片机读取到的电平可能会在高低电平之间快速变化。如果不进行消抖处理，程序可能会误认为按键被多次按下。

软件消抖的基本思想是：当检测到按键电平变化后，先延时一小段时间，例如 10ms 到 20ms，然后再次读取按键电平。如果两次读取结果一致，才认为按键状态有效。

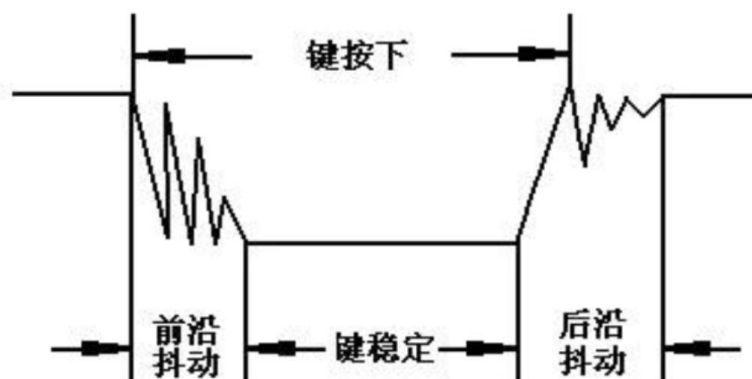


图 3.3 按键抖动波形

3.2.4 蜂鸣器控制原理

本实验使用无源蜂鸣器。无源蜂鸣器本身不能直接发声，必须由单片机给它提供周期性变化的方波信号。

当 GPIO 输出高电平一段时间，再输出低电平一段时间，并不断重复时，蜂鸣器就会振动发声。

如果高低电平翻转间隔较长，蜂鸣器频率较低，声音较低沉。

如果高低电平翻转间隔较短，蜂鸣器频率较高，声音较尖锐。

本实验不使用 TIM 定时器和 PWM 功能，仅通过 GPIO 翻转和软件延时产生蜂鸣器声音。需要注意的是，软件延时产生的声音频率不够精确，但足以满足本实验的教学要求。

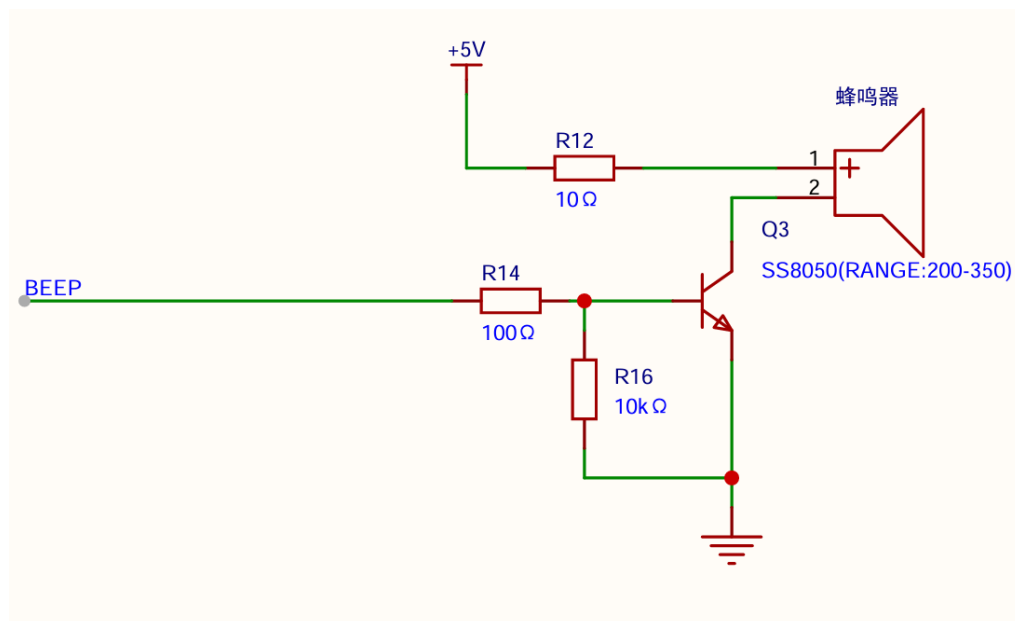


图 3.4 蜂鸣器模块原理图

3.3 实验步骤

(1) 编写蜂鸣器部分代码

第一步，按实验二里面添加 LED.c 和 LED.h 的步骤，在 HARDWARE 里面添加蜂鸣器的 beep.c ,beep.h 如图 3.5 所示，注意这里新添加的文件在 USER 文件夹下面，可以通过另存为，加到 HARDWARE 文件夹中。

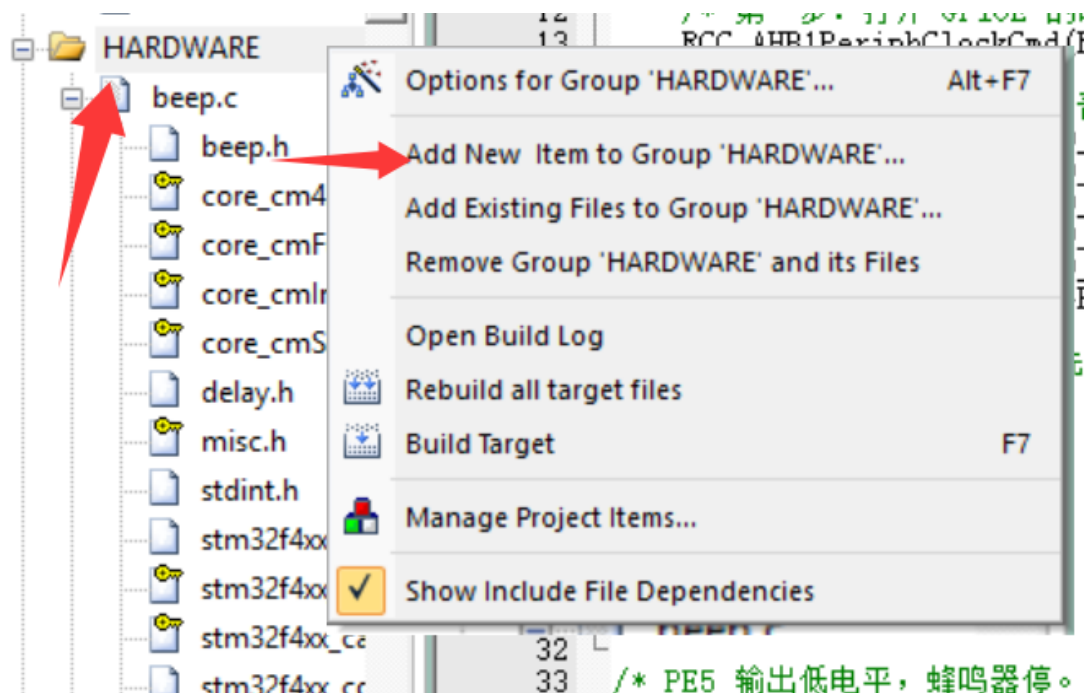


图 3.5 新建 beep.c 和 beep.h 文件

(2) 编写 beep.c 和 beep.h 代码

在 HARDWARE/BEEP 文件夹中新建 beep.c 文件。首先在文件开头包含两个头文件：#include "beep.h", #include "delay.h"

其中，beep.h 提供蜂鸣器端口、引脚和函数声明；delay.h 提供微秒级延时函数 delay_us(), 用于产生不同频率的蜂鸣器声音。

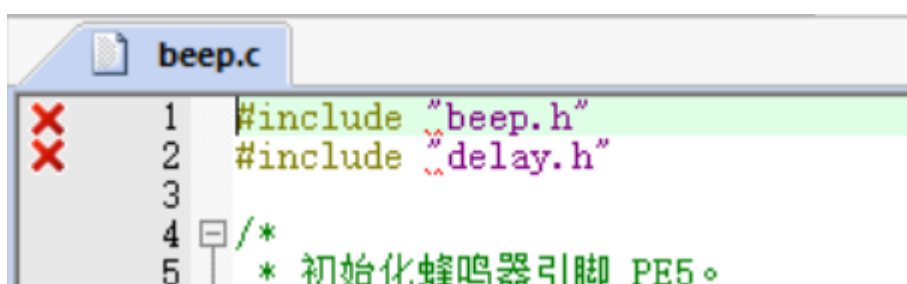


图 3.6 编写头文件

初始化蜂鸣器引脚

在 beep.c 中编写 BEEP_Init() 函数。该函数主要完成三件事。第一步，定义 GPIO 初始化结构体：GPIO_InitTypeDef GPIO_InitStructure;

第二步，打开 GPIOE 的时钟。因为蜂鸣器连接在 PE5，而 PE5 属于 GPIOE，

所以必须先使能 GPIOE 时钟：RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOE, ENABLE);

STM32 的外设在使用前必须先打开对应时钟。如果没有打开 GPIOE 时钟，即使后面配置了 PE5，单片机也无法正常控制该引脚。

第三步，将 PE5 配置为普通推挽输出模式：调用：BEEP_Off();使蜂鸣器初始化后保持关闭状态，避免开发板上电后蜂鸣器误响。

```
8 void BEEP_Init(void)
9 {
10     GPIO_InitTypeDef GPIO_InitStructure;
11
12     /* 第一步：打开 GPIOE 的时钟。PE5 属于 GPIOE。 */
13     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOE, ENABLE);
14
15     /* 第二步：把 PE5 配置成普通推挽输出。 */
16     GPIO_InitStructure.GPIO_Pin = BEEP_PIN;
17     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
18     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
19     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
20     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_DOWN;
21     GPIO_Init(BEEP_PORT, &GPIO_InitStructure);
22
23     /* 第三步：初始化完成后先关闭蜂鸣器，避免上电乱响。 */
24     BEEP_Off();
25 }
```

图 3.7 初始化蜂鸣器引脚

编写蜂鸣器开关函数

蜂鸣器连接在 PE5，根据硬件连接关系，PE5 输出高电平时蜂鸣器响，PE5 输出低电平时蜂鸣器停。

GPIO_SetBits() 的作用是将指定 GPIO 引脚置为高电平，GPIO_ResetBits() 的作用是将指定 GPIO 引脚置为低电平

```

/* PE5 输出高电平，蜂鸣器响。 */
void BEEP_On(void)
{
    GPIO_SetBits(BEEP_PORT, BEEP_PIN);
}

/* PE5 输出低电平，蜂鸣器停。 */
void BEEP_Off(void)
{
    GPIO_ResetBits(BEEP_PORT, BEEP_PIN);
}

```

图 3.8 蜂鸣器开关函数

编写调频函数

无源蜂鸣器不能只靠一直输出高电平持续发出正常声音，而是需要单片机不断输出高低变化的电平。

函数如下：

```

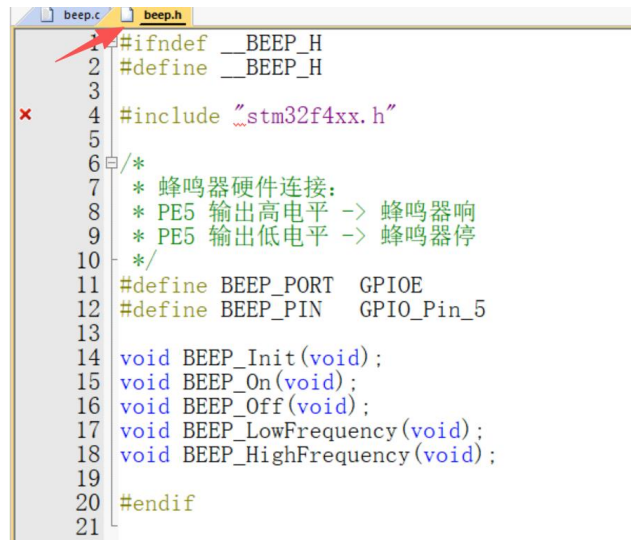
void BEEP_LowFrequency(void)
{
    BEEP_On();
    delay_us(1000);
    BEEP_Off();
    delay_us(1000);
}

/*
 * 高频声音。
 * 延时较短，翻转较快，声音偏高。
 */
void BEEP_HighFrequency(void)
{
    BEEP_On();
    delay_us(50);
    BEEP_Off();
    delay_us(50);
}

```

图 3.8 调频函数

编写 beep.h 文件



```

1 #ifndef __BEEP_H
2 #define __BEEP_H
3
4 #include "stm32f4xx.h"
5
6 /*
7  * 蜂鸣器硬件连接:
8  *  PE5 输出高电平 -> 蜂鸣器响
9  *  PE5 输出低电平 -> 蜂鸣器停
10 */
11 #define BEEP_PORT  GPIOE
12 #define BEEP_PIN   GPIO_Pin_5
13
14 void BEEP_Init(void);
15 void BEEP_On(void);
16 void BEEP_Off(void);
17 void BEEP_LowFrequency(void);
18 void BEEP_HighFrequency(void);
19
20 #endif
21

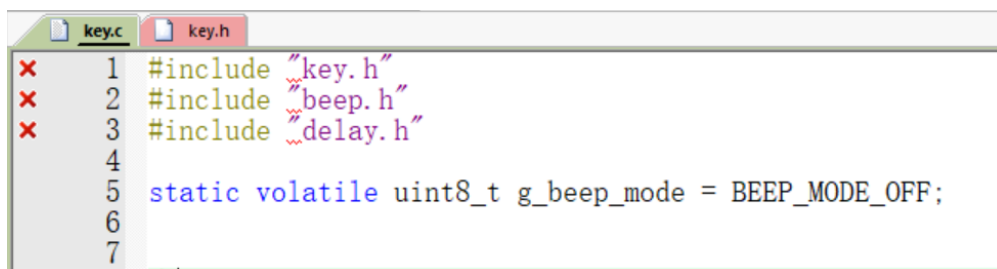
```

图 3.9 beep.h 文件

(2) 编写按键外部中断部分代码

先新建 key.c 和 key.h 文件（同上），然后加入头文件，其中，key.h 提供按键引脚和函数声明，beep.h 提供蜂鸣器控制函数。接着定义蜂鸣器工作模式，该变量用于记录当前蜂鸣器应该处于哪种模式。

这里使用 volatile 是因为该变量可能会在中断相关逻辑中被修改。加上 volatile 后，可以提醒编译器不要随意优化这个变量，保证程序每次读取时都能获得最新状态。程序初始时将其设置为 BEEP_MODE_OFF，表示蜂鸣器默认关闭。



```

1 #include "key.h"
2 #include "beep.h"
3 #include "delay.h"
4
5 static volatile uint8_t g_beep_mode = BEEP_MODE_OFF;
6
7

```

图 3.10 key.c 头文件

编写按键状态读取函数，该函数读取按键的输入电平。如果读取结果为 Bit_RESET，说明引脚为低电平，即 KEY1 或 KEY2 被按下，函数返回 1；否则返回 0，表示按键松开。


```

/*
 * 读取 KEY1 状态。
 * 返回 1: 按键按下
 * 返回 0: 按键松开
 */
static uint8_t KEY1_IsPressed(void)
{
    if (GPIO_ReadInputDataBit(KEY1_PORT, KEY1_PIN) == Bit_RESET)
    {
        return 1;
    }

    return 0;
}

/*
 * 读取 KEY2 状态。
 * 返回 1: 按键按下
 * 返回 0: 按键松开
 */
static uint8_t KEY2_IsPressed(void)
{
    if (GPIO_ReadInputDataBit(KEY2_PORT, KEY2_PIN) == Bit_RESET)
    {
        return 1;
    }

    return 0;
}

```

图 3.11 按键状态读取函数

编写蜂鸣器模式更新函数

该函数的判断逻辑如下：

首先判断 KEY2 是否按下。如果 KEY2 按下，则蜂鸣器模式设置为高频：

如果 KEY2 没有按下，再判断 KEY1 是否按下。如果 KEY1 按下，则蜂鸣器模式设置为低频：

如果两个按键都没有按下，则蜂鸣器模式设置为关闭，并立即调用 BEEP_Off() 关闭蜂鸣器。

这里需要注意：如果两个按键同时按下，程序会优先判断 KEY2，因此高频蜂鸣优先级高于低频蜂鸣。

```

/*
 * 根据两个按键的状态，决定蜂鸣器应该发出哪种声音。
 * 如果两个按键同时按下，高频优先。
 */
static void KEY_UpdateBeepMode(void)
{
    if (KEY2_IsPressed())
    {
        g_beep_mode = BEEP_MODE_HIGH;
    }
    else if (KEY1_IsPressed())
    {
        g_beep_mode = BEEP_MODE_LOW;
    }
    else
    {
        g_beep_mode = BEEP_MODE_OFF;
        BEEP_Off();
    }
}

uint8_t KEY_GetBeepMode(void)
{
    return g_beep_mode;
}

```

图 3.12 按键状态抢先判断

配置按键 GPIO 和外部中断

在 key.c 中，按键初始化函数为 KEY_Init()。该函数是本实验的重点，学生需要理解每一步配置的作用。

定义初始化结构体，GPIO_InitTypeDef GPIO_InitStructure; EXTI_InitTypeDef EXTI_InitStructure; NVIC_InitTypeDef NVIC_InitStructure;

其中，GPIO_InitTypeDef 用于配置按键引脚为输入模式；EXTI_InitTypeDef 用于配置外部中断线；NVIC_InitTypeDef 用于配置中断通道和优先级。

打开 GPIOC、GPIOE 和 SYSCFG 时钟，

RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC | RCC_AHB1Periph_GPIOE, ENABLE);
RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);

这里需要打开三个时钟，GPIOC 时钟用于 PC13，也就是 KEY1。GPIOE 时钟用于 PE4，也就是 KEY2，同时蜂鸣器 PE5 也在 GPIOE 上。SYSCFG 时钟用于配置 GPIO 和 EXTI 中断线之间的连接关系。如果忘记打开 SYSCFG 时钟，EXTI 端口映射可能无法正常工作。

```

/*
 * 初始化 PC13 和 PE4 外部中断。
 *
 * 配置结果：
 * PC13/PE4 高电平 -> 松开
 * PC13/PE4 低电平 -> 按下
 * 电平从高到低或从低到高变化时，都会进入对应的中断函数。
 */
void KEY_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    EXTI_InitTypeDef EXTI_InitStructure;
    NVIC_InitTypeDef NVIC_InitStructure;

    /* 第一步：打开 GPIOC、GPIOE 和 SYSCFG 的时钟。 */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC | RCC_AHB1Periph_GPIOE, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);

    /* 第二步：PC13 配置为上拉输入。没按下时稳定为高电平。 */
    GPIO_InitStructure.GPIO_Pin = KEY1_PIN;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(KEY1_PORT, &GPIO_InitStructure);

    /* 第三步：PE4 也配置为上拉输入。 */
    GPIO_InitStructure.GPIO_Pin = KEY2_PIN;
    GPIO_Init(KEY2_PORT, &GPIO_InitStructure);

    /* 第四步：告诉 EXTI，13 号中断线连接到 PC13，4 号中断线连接到 PE4。 */
    SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOC, EXTI_PinSource13);
    SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOE, EXTI_PinSource4);

    /* 第五步：配置 EXTI13，上升沿和下降沿都触发。 */
    EXTI_InitStructure.EXTI_Line = EXTI_Line13;
    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising_Falling;
    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
    EXTI_Init(&EXTI_InitStructure);

    /* 第五步：配置 EXTI13，上升沿和下降沿都触发。 */
    EXTI_InitStructure.EXTI_Line = EXTI_Line13;
    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising_Falling;
    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
    EXTI_Init(&EXTI_InitStructure);

    /* 第六步：配置 EXTI4，上升沿和下降沿都触发。 */
    EXTI_InitStructure.EXTI_Line = EXTI_Line4;
    EXTI_Init(&EXTI_InitStructure);

    /* 第七步：在 NVIC 中打开 EXTI15_10 中断通道。PC13 会用到它。 */
    NVIC_InitStructure.NVIC_IRQChannel = EXTI15_10_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 2;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 2;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);

    /* 第八步：在 NVIC 中打开 EXTI4 中断通道。PE4 会用到它。 */
    NVIC_InitStructure.NVIC_IRQChannel = EXTI4_IRQn;
    NVIC_Init(&NVIC_InitStructure);

    KEY_UpdateBeepMode();
}
/*

```

图 3.13 按键和中断初始化

配置 GPIO 与 EXTI 中断线映射

STM32F407 的外部中断线不是自动知道连接到哪个 GPIO 端口的，需要通过 SYSCFG_EXTILineConfig() 进行映射。KEY1 接 PC13，因此需要将 EXTI13 映射到 GPIOC。

其中，EXTI_Line13 表示配置第 13 号外部中断线。EXTI_Mode_Interrupt 表示中断模式。EXTI_Trigger_Rising_Falling 表示上升沿和下降沿都触发。EXTI_LineCmd = ENABLE 表示使能该中断线。

本实验需要检测按键按下和松开两个动作。按键按下时，电平从高变低，是下降沿；按键松开时，电平从低变高，是上升沿。因此这里配置为上升沿和下降沿均触发。

编写 PE4 中断服务函数

PE4 对应 EXTI4，因此中断服务函数为，该函数的处理逻辑与 PC13 类似，先判断 EXTI4 是否产生中断；然后执行软件消抖；接着更新蜂鸣器模式；最后清除 EXTI4 中断挂起标志位。

```
/*
 * PC13 属于 EXTI13。
 * EXTI10 到 EXTI15 共用同一个中断函数：EXTI15_10_IRQHandler。
 */
void EXTI15_10_IRQHandler(void)
{
    /* 先判断这次中断是不是 EXTI13 产生的。 */
    if (EXTI_GetITStatus(EXTI_Line13) != RESET)
    {
        /* 简单软件消抖：延时 20ms，等电平稳定后再读取。 */
        delay_ms(20);
        KEY_UpdateBeepMode();

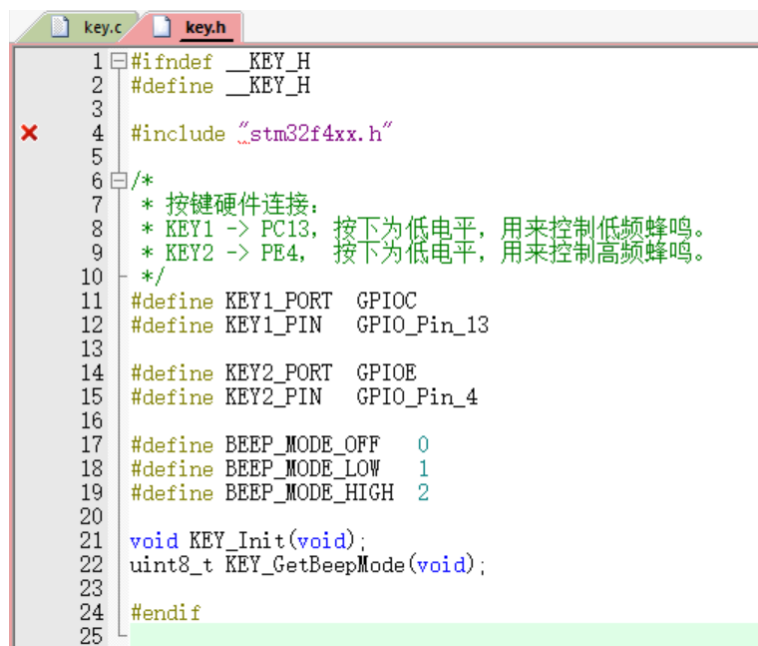
        /* 清除中断标志位。否则会一直重复进入中断。 */
        EXTI_ClearITPendingBit(EXTI_Line13);
    }
}

/*
 * PE4 属于 EXTI4，所以使用 EXTI4_IRQHandler。
 */
void EXTI4_IRQHandler(void)
{
    if (EXTI_GetITStatus(EXTI_Line4) != RESET)
    {
        /* 简单软件消抖：延时 20ms，等电平稳定后再读取。 */
        delay_ms(20);
        KEY_UpdateBeepMode();

        EXTI_ClearITPendingBit(EXTI_Line4);
    }
}
```

图 3.14 外部中断函数

编写 key.h 函数



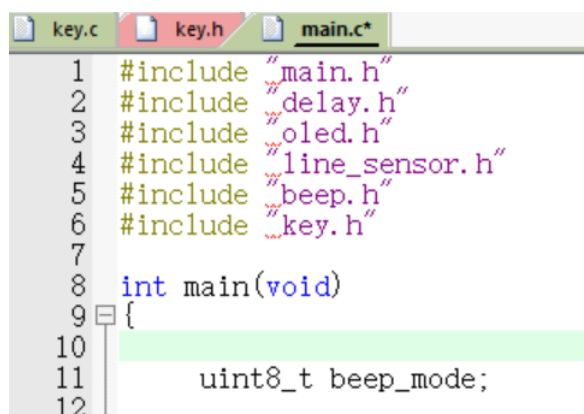
```
1 #ifndef __KEY_H
2 #define __KEY_H
3
4 #include "stm32f4xx.h"
5
6 /*
7  * 按键硬件连接:
8  * KEY1 -> PC13, 按下为低电平, 用来控制低频蜂鸣。
9  * KEY2 -> PE4, 按下为低电平, 用来控制高频蜂鸣。
10 */
11 #define KEY1_PORT  GPIOC
12 #define KEY1_PIN  GPIO_Pin_13
13
14 #define KEY2_PORT  GPIOE
15 #define KEY2_PIN  GPIO_Pin_4
16
17 #define BEEP_MODE_OFF  0
18 #define BEEP_MODE_LOW  1
19 #define BEEP_MODE_HIGH 2
20
21 void KEY_Init(void);
22 uint8_t KEY_GetBeepMode(void);
23
24 #endif
25
```

图 3.15 key.h 文件

(3) 编写 main.c 主函数

在 main.c 中，首先包含实验需要的头文件：`#include "main.h"`，`#include "delay.h"`，`#include "beep.h"`，`#include "key.h"`。其中，`delay.h` 用于提供蜂鸣器频率控制所需的延时函数；`beep.h` 提供蜂鸣器初始化和发声函数；`key.h` 提供按键初始化和蜂鸣器模式读取函数。

定义蜂鸣器模式变量，在主函数中定义变量 `uint8_t beep_mode`；该变量用于保存当前蜂鸣器模式。



```
1 #include "main.h"
2 #include "delay.h"
3 #include "oled.h"
4 #include "line_sensor.h"
5 #include "beep.h"
6 #include "key.h"
7
8 int main(void)
9 {
10
11     uint8_t beep_mode;
12
13 }
```

图 3.16 main.c 头文件

```

delay_init(168);
BEEP_Init();
KEY_Init();

```

图 3.16 各个模块初始化

KEY_Init(), 函数会完成 PC13、PE4 的上拉输入配置、EXTI 中断线配置、SYSCFG 映射以及 NVIC 中断使能。

完成这一步后, 当 KEY1 或 KEY2 被按下或松开时, 单片机会自动进入对应的外部中断服务函数。

```

while (1)
{
    beep_mode = KEY_GetBeepMode();
    if (beep_mode == BEEP_MODE_LOW)
    {
        BEEP_LowFrequency();
    }
    else if (beep_mode == BEEP_MODE_HIGH)
    {
        BEEP_HighFrequency();
    }
    else
    {
        BEEP_Off();
    }
}

#ifdef USE_FULL_ASSERT
void assert_failed(uint8_t* file, uint32_t line)
{
    while (1)
    {
    }
}
#endif

```

图 3.16 主循环代码

下载程序到开发板

将 J-Link 与 STM32F407VET6 开发板正确连接。

确认 Keil 中 Debug 选项已经选择 J-Link Debugger, 并且能够识别芯片。

然后点击 Download 按钮, 将程序下载到开发板中。

下载成功后，开发板复位运行程序。

实验四 红外避障与循迹

4.1 实验目的

本节实验主要学习 32GPIO 端口作为输入的相关操作，希望同学们了解并掌握 STM32F407 的 IO 口作为输入使用的方法，从而后续过程中可以实现对红外对管传感器的灵活运用。

4.2 实验简介

4.2.1 红外对管传感器

(1) 原理：

发射管发射出一定频率的红外线，当检测方向遇到障碍物（反射面）时，红外线反射回来被接收管接收，经过比较器电路处理之后，输出指示灯会亮起，同时信号输出接口输出数字信号（一个低电平信号）。



图 4.1 红外对管传感器图

(2) 运用：

①红外避障：经常安装在小车前端两侧，用于检测障碍物与小车之间的距离，通过红外发射/接收电路工作，接收到的红外光强度会影响电流大小，从而判断障碍物的存在与否，实现避障功能。

②红外循迹：通常安装在小车底盘两侧，用于检测黑线与小车的相对位置，黑色

能够吸收红外线而白色不行，即红外扫到白线时会返回一个高电平，保持小车在轨道上运行，所以可用作黑白循迹，还可以用作光电开关等；

4.2.2 硬件设计

如图 4.2 是一路红外循迹传感器检测电路，作用是把地面黑白反射差异转换成单片机可以识别的高低电平信号。

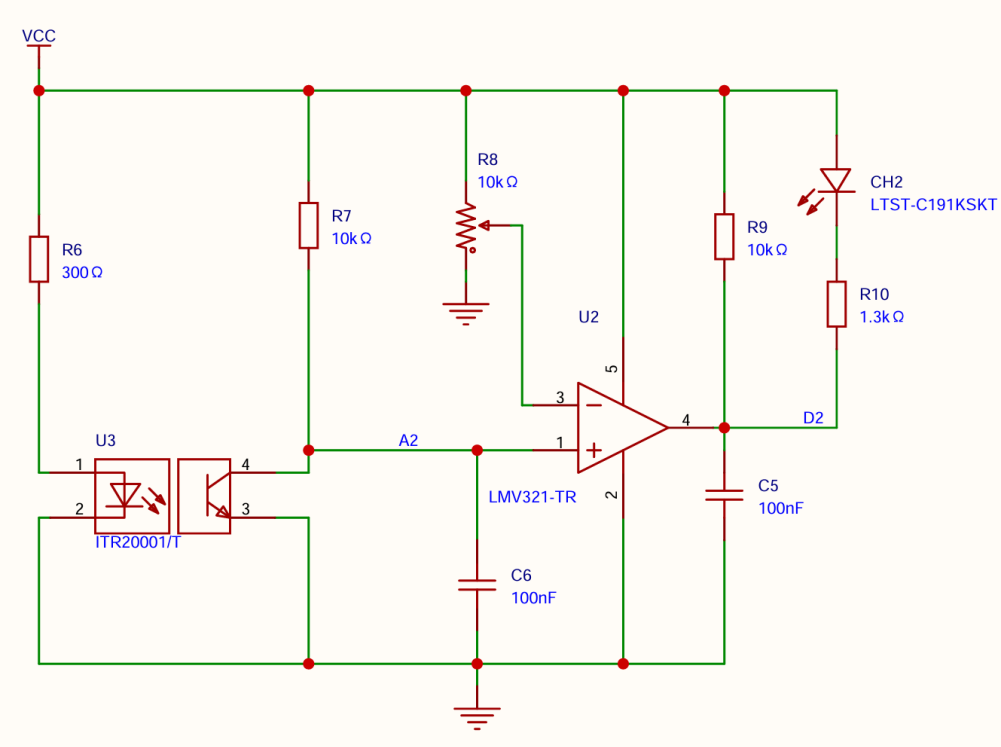


图 4.2 一路红外传感器原理图

电路左侧的 U3（ITR20001/T）是红外对管，内部包含一个红外发射管和一个红外接收管。红外发射管由 VCC → R6 → 发射管 → GND 供电，R6 起限流作用，使红外管持续向地面发射红外光。接收管用于接收地面反射回来的红外光，反射光强弱会改变接收管的导通程度。

中间的 R7 和 C6 组成接收信号处理部分。R7 是上拉电阻，接收管导通强弱会改变 A2 点电压；C6 起滤波作用，减少信号抖动和干扰。当地面反射强时，接收管导通增强，A2 电压会发生变化；当地面反射弱时，A2 电压也会随之变化。

右侧的 U2（LMV321）在这里相当于比较器使用。它把 A2 点的传感器信号与由 R8 电位器调节出来的参考电压进行比较。R8 的作用是调节检测灵敏度，也就是

设置黑线和白底的判断阈值。

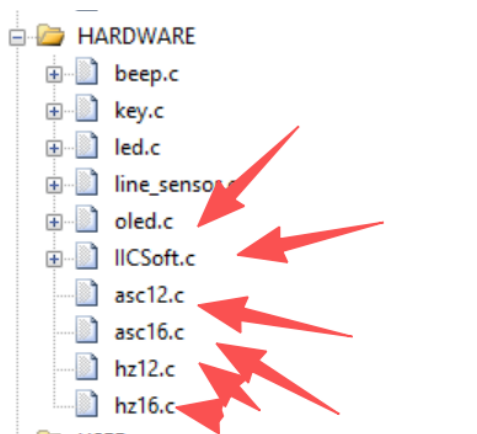
比较后的输出信号从 D2 输出，可接到 STM32 的 GPIO 输入口。D2 输出高电平或低电平，单片机就可以判断当前传感器检测到的是黑线还是白底。

右侧的 CH2 和 R10 是状态指示灯电路，用来显示当前输出状态。R10 是限流电阻，防止指示灯电流过大。R9 和 C5 用于输出端上拉和滤波，使 D2 输出更加稳定。

4.3 实验过程

IO 操作步骤很简单，操作步骤概括为：

- 1) 使能 IO 口时钟。调用函数为 `RCC_APB2PeriphClockCmd()`。
- 2) 初始化 IO 参数。调用函数 `GPIO_Init()`;与上一节实验不同的是其模式的配置。
- 3) 操作 IO。操作 IO 的方法就是需要用调用上面提到的各个函数。
 - (1) 复制之前配置好的工程模板，新建 HARDWARE 文件夹。
 - (2) 添加 OLED 相关的文件如下图所示



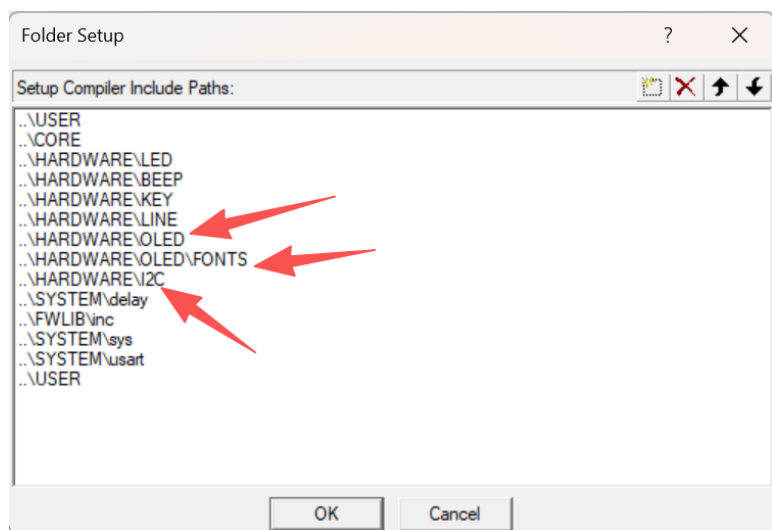
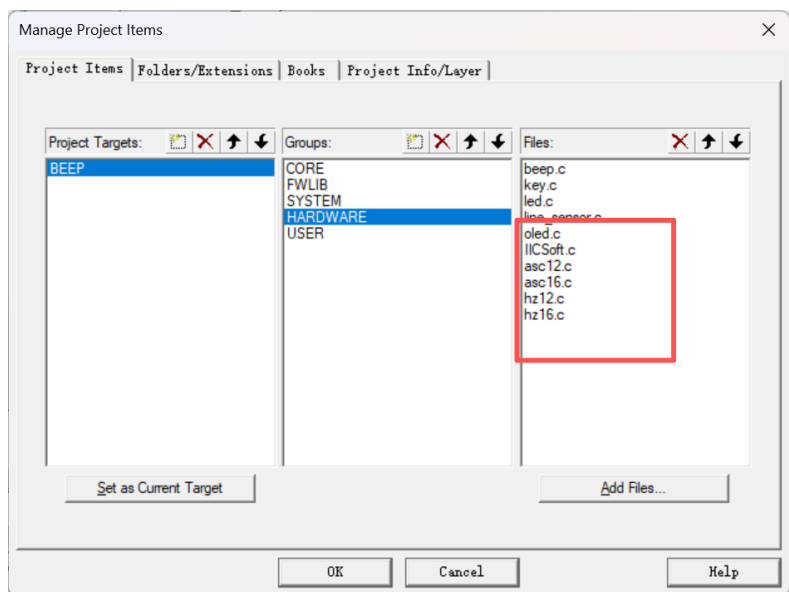


图 4.3 添加 OLED 需要的相关文件

(3) 编写红外传感器文件 line_sensor.c 和 line_sensor.h

本实验通过 STM32F407 的 GPIO 输入功能实现八路红外循迹传感器状态采集。八路循迹传感器分别对应 CH1 至 CH8，每一路传感器输出一个数字电平信号，单片机通过读取对应 GPIO 引脚的电平状态判断该路传感器是否检测到黑线或有效信号。

本程序主要包括三个部分：八路传感器 GPIO 引脚映射、传感器 GPIO 初始化、单路及八路传感器状态读取。

```

1  #include "line_sensor.h"
2
3  typedef struct
4  {
5      GPIO_TypeDef *port;
6      uint16_t pin;
7  } LINE_GPIO_T;
8
9  /* CH1 到 CH8 对应的 GPIO 端口和引脚, 顺序和原理图上的 LINE_CH1~LINE_CH8 保持一致。 */
10 static const LINE_GPIO_T g_line_gpio[LINE_SENSOR_COUNT] =
11 {
12     {LINE_CH1_PORT, LINE_CH1_PIN},
13     {LINE_CH2_PORT, LINE_CH2_PIN},
14     {LINE_CH3_PORT, LINE_CH3_PIN},
15     {LINE_CH4_PORT, LINE_CH4_PIN},
16     {LINE_CH5_PORT, LINE_CH5_PIN},
17     {LINE_CH6_PORT, LINE_CH6_PIN},
18     {LINE_CH7_PORT, LINE_CH7_PIN},
19     {LINE_CH8_PORT, LINE_CH8_PIN}
20 };
21
22 void LINE_Init(void)
23 {
24     GPIO_InitTypeDef GPIO_InitStructure;
25
26     /* 8 路光电管分布在 GPIOA、GPIOB、GPIOC, 使用前先打开这三个端口的时钟。 */
27     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA |
28                             RCC_AHB1Periph_GPIOB |
29                             RCC_AHB1Periph_GPIOC, ENABLE);
30
31     /* 光电管输出端接到单片机输入引脚; 无信号时为高电平, 所以这里配置上拉输入。 */
32     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
33     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
34     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
35     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
36
37     GPIO_InitStructure.GPIO_Pin = LINE_CH5_PIN | LINE_CH6_PIN;
38     GPIO_Init(GPIOA, &GPIO_InitStructure);
39
40     GPIO_InitStructure.GPIO_Pin = LINE_CH1_PIN | LINE_CH2_PIN;
41     GPIO_Init(GPIOB, &GPIO_InitStructure);
42
43     GPIO_InitStructure.GPIO_Pin = LINE_CH3_PIN | LINE_CH4_PIN |
44                                     LINE_CH7_PIN | LINE_CH8_PIN;
45     GPIO_Init(GPIOC, &GPIO_InitStructure);
46 }

```

```

uint8_t LINE_ReadChannel(uint8_t channel)
{
    const LINE_GPIO_T *line;

    /* channel 的有效范围是 1~8, 超出范围直接返回无信号。 */
    if ((channel == 0) || (channel > LINE_SENSOR_COUNT))
    {
        return 0;
    }

    line = &g_line_gpio[channel - 1];

    /* 低电平表示检测到信号, 返回 1; 高电平表示没有信号, 返回 0。 */
    if (GPIO_ReadInputDataBit(line->port, line->pin) == Bit_RESET)
    {
        return 1;
    }

    return 0;
}

uint8_t LINE_ReadAll(void)
{
    uint8_t i;
    uint8_t state = 0;

    /* 返回值的 bit0~bit7 分别表示 CH1~CH8, bit 为 1 表示该路检测到信号。 */
    for (i = 0; i < LINE_SENSOR_COUNT; i++)
    {
        if (LINE_ReadChannel(i + 1))
        {
            state |= (uint8_t)(1U << i);
        }
    }

    return state;
}

```

图 4.4 line_sensor.c 文件

本实验中规定：低电平表示检测到信号，返回 1；高电平表示未检测到信号，返回 0。

因此，当 GPIO 读取结果为 Bit_RESET 时，说明该路传感器输出低电平，即检测到黑线或有效信号，函数返回 1；否则返回 0。

LINE_ReadAll() 函数用于一次性读取 CH1 到 CH8 的状态，并将八路状态组合成一个 8 位数据返回。

其中，state 用于保存最终的八路传感器状态。返回值的 bit0 到 bit7 分别对应 CH1 到 CH8：bit0 → CH1，bit1 → CH2，bit2 → CH3，bit3 → CH4，bit4 → CH5，bit5 → CH6，bit6 → CH7，bit7 → CH8



```
1 #ifndef __LINE_SENSOR_H
2 #define __LINE_SENSOR_H
3
4 #include "stm32f4xx.h"
5 #include <stdint.h>
6
7 /*
8  * 8 路光电管信号说明：
9  * 低电平 -> 检测到信号
10  * 高电平 -> 没有检测到信号
11  */
12 #define LINE_SENSOR_COUNT 8
13
14 /* 原理图引脚对应关系：LINE_CH1 -> PB0。 */
15 #define LINE_CH1_PORT  GPIOB
16 #define LINE_CH1_PIN  GPIO_Pin_0
17
18 /* LINE_CH2 -> PB1。 */
19 #define LINE_CH2_PORT  GPIOB
20 #define LINE_CH2_PIN  GPIO_Pin_1
21
22 /* LINE_CH3 -> PC4。 */
23 #define LINE_CH3_PORT  GPIOC
24 #define LINE_CH3_PIN  GPIO_Pin_4
25
26 /* LINE_CH4 -> PC5。 */
27 #define LINE_CH4_PORT  GPIOC
28 #define LINE_CH4_PIN  GPIO_Pin_5
29
30 /* LINE_CH5 -> PA4。 */
31 #define LINE_CH5_PORT  GPIOA
32 #define LINE_CH5_PIN  GPIO_Pin_4
33
34 /* LINE_CH6 -> PA5。 */
35 #define LINE_CH6_PORT  GPIOA
36 #define LINE_CH6_PIN  GPIO_Pin_5
37
38 /* LINE_CH7 -> PC3。 */
39 #define LINE_CH7_PORT  GPIOC
40 #define LINE_CH7_PIN  GPIO_Pin_3
41
```

```

/* LINE_CH6 -> PA5。 */
#define LINE_CH6_PORT GPIOA
#define LINE_CH6_PIN   GPIO_Pin_5

/* LINE_CH7 -> PC3。 */
#define LINE_CH7_PORT GPIOC
#define LINE_CH7_PIN   GPIO_Pin_3

/* LINE_CH8 -> PC2。 */
#define LINE_CH8_PORT GPIOC
#define LINE_CH8_PIN   GPIO_Pin_2

/* 初始化 GPIO 输入。 */
void LINE_Init(void);

/* 读取单路光电管, channel 范围 1~8, 返回 1 表示检测到信号。 */
uint8_t LINE_ReadChannel(uint8_t channel);

/* 一次读取 8 路状态, bit0~bit7 对应 CH1~CH8。 */
uint8_t LINE_ReadAll(void);

#endif

```

图 4.4 line_sensor.h 文件

(4) 编写 main.c

主函数程序设计说明

本程序主要实现 8 路光电循迹传感器状态采集与 OLED 实时显示。程序通过 LINE_ReadAll() 一次性读取 CH1 到 CH8 的检测状态，并将每一路状态转换为字符 0 或 1，最后显示在 OLED 屏幕上，方便学生直观看到每一路传感器是否检测到黑线或有效信号。

程序开头包含了实验所需的头文件：`#include "main.h", #include "delay.h", #include "oled.h", #include "line_sensor.h"`

第一步，`delay_init(168)` 用于初始化系统延时函数。STM32F407 的主频通常为 168MHz，因此这里传入参数 168。后续程序中的 `delay_ms(100)` 需要依赖该初始化函数。

第二步，`OLED_Init()` 用于初始化 OLED 显示屏。只有完成 OLED 初始化后，后续的清屏、绘图和显示字符串函数才能正常工作。

第三步，`LINE_Init()` 用于初始化 8 路光电循迹传感器输入引脚。该函数会将 CH1 到 CH8 对应的 GPIO 引脚配置为输入模式，使 STM32 能够读取每一路传感器输出的高低电平。

程序进入 while(1) 后，会不断执行传感器状态读取和 OLED 刷新显示。

line_state = LINE_ReadAll();该函数会一次性读取 CH1 到 CH8 的状态，并将结果保存到一个 8 位变量 line_state 中。

```
line_sensor.c line_sensor.h main.c
1 #include "main.h"
2 #include "delay.h"
3 #include "oled.h"
4 #include "line_sensor.h"
5 #include "beep.h"
6 #include "key.h"
7
8 int main(void)
9 {
10     uint8_t i;
11     uint8_t line_state;
12     char line1[] = "CH: 1 2 3 4";
13     char line2[] = "S : 0 0 0 0";
14     char line3[] = "CH: 5 6 7 8";
15     char line4[] = "S : 0 0 0 0";
16     // uint8_t beep_mode;
17
18     /*
19      * 1. 初始化系统延时。
20      * 2. 初始化 OLED 屏幕。
21      * 3. 初始化 8 路光电管输入。
22      */
23     delay_init(168);
24     OLED_Init();
25     LINE_Init();
26
27     /*
28      * 蜂鸣器和按键功能先保留，当前先注释掉。
29      * 后面需要恢复蜂鸣器时，取消下面初始化和主循环里的蜂鸣器代码注释即可。
30      */
31
32     // BEEP_Init();
33     // KEY_Init();
34
35
36     while (1)
37     {
38         /* 第一步：读取 8 路光电管状态。低电平表示检测到信号，显示为 1。 */
39         line_state = LINE_ReadAll();
40
41         /* 第二步：把读取结果填到要显示的字符串里。 */
42         for (i = 0; i < 4; i++)
43         {
44             line2[4 + i * 2] = (line_state & (1U << i)) ? '1' : '0';
45             line4[4 + i * 2] = (line_state & (1U << (i + 4))) ? '1' : '0';
46         }
47
48         /* 第三步：刷新 OLED，把 8 路检测结果显示出来。 */
49         OLED_StartDraw();
50         OLED_ClrScr(0x00);
51         OLED_DisStr(0, 0, line1, &tFont12);
52         OLED_DisStr(0, 14, line2, &tFont12);
53         OLED_DisStr(0, 32, line3, &tFont12);
54         OLED_DisStr(0, 46, line4, &tFont12);
55         OLED_EndDraw();
56
57         delay_ms(100);
58     }
59 }
60
61 #ifdef USE_FULL_ASSERT
62 void assert_failed(uint8_t* file, uint32_t line)
63 {
64     while (1)
65     {
66     }
67 }
68 #endif
```

图 4.4 main.c 文件

下载程序到开发板

将 J-Link 与 STM32F407VET6 开发板正确连接。

确认 Keil 中 Debug 选项已经选择 J-Link Debugger，并且能够识别芯片。

然后点击 Download 按钮，将程序下载到开发板中。

程序下载成功后，OLED 屏幕应显示 4 行内容：



图 4.5 屏幕打印结果

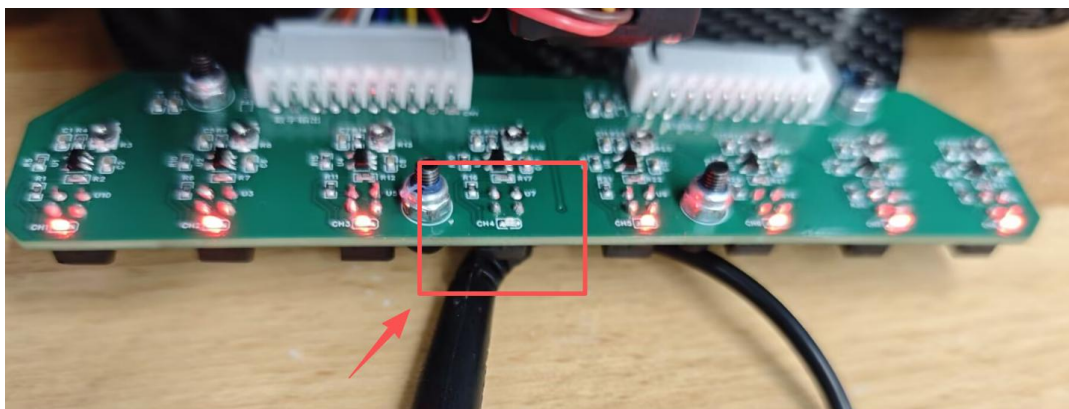


图 4.6 传感器遮挡效果

当通道 4 被挡住时，屏幕上通道 4 显示 0。

附加题

题目：通过八路光电管不同通道输入，对应蜂鸣器种不同输出音效

需要在 beep.c 中书写一个函数 void BEEP_LineSensorTone(uint8_t line_state)，如下

图 4.7 所示，使得其主函数中能被调用。

```
/*
 * 根据 8 路光电管状态发声。
 * line_state 为 0x01~0x80 时，分别对应 CH1~CH8。
 */
void BEEP_LineSensorTone(uint8_t line_state)
{
    uint16_t i;
```

图 4.7 BEEP_LineSensorTone(uint8_t line_state)函数

```
/* 第四步：根据 8 路光电管状态，调用蜂鸣器模块输出对应通道的频率。 */
BEEP_LineSensorTone(line_state);
...
}
```

图 4.9 主循环里面调用